De Problemas Reales a Código Python:
Formulación de Programación Lineal y Resolución con PuLP

Investigación de Operaciones · Universidad Juárez del Estado de Durango · Prof. Mario

Para el estudiante. Esta guía te enseña a *leer* un problema real y *construir* el modelo de programación lineal desde cero. La parte difícil no es el código: es la traducción de palabras a ecuaciones. Léela completa antes del examen. El código que necesitas está aquí; puedes traerlo preparado desde casa.

1. El Salto que Hay que Dar

En casi todos los cursos de matemáticas anteriores, los problemas llegan ya formulados: "Maximiza $3x_1 + 5x_2$ sujeto a..." Alguien ya hizo la traducción por ti. Tu trabajo era ejecutar el método. Eso es valioso, pero es solo la mitad de la habilidad.

Esta unidad te pide la otra mitad: te dan un párrafo que describe una situación real — una cooperativa artesanal, un rancho ganadero, una farmacia — y tú tienes que construir el modelo matemático desde cero. Solo después de construirlo puedes resolverlo.

Ese paso de construcción se llama **formulación**, y es la habilidad más importante de la programación lineal. En la vida profesional, nadie te entrega el modelo. Te dan el problema.

El cambio de mentalidad.
Antes: "aquí están las ecuaciones, resuélvelas."
Ahora: "aquí está la situación, construye las ecuaciones y luego resuélvelas." El segundo paso incluye al primero.

Hay dos tipos de conocimiento muy distintos: saber *ejecutar* un algoritmo y saber *modelar* una situación. El simplex te enseñó a ejecutar. Esta unidad te enseña a modelar.

2. Las Cuatro Preguntas de la Formulación

Todo problema de PL, por complicado que parezca, se formula respondiendo cuatro preguntas. Siempre las mismas, siempre en orden.

2.1. Pregunta 1: ¿Qué estás decidiendo?

Identifica las **variables de decisión**: las cantidades que *tú controlas*. ¿Qué puede elegir el tomador de decisiones?

Señales en el texto que indican una variable: "¿cuánto producir?", "¿cuántas unidades comprar?", "¿qué cantidad de cada ingrediente?", "¿cuántos turnos asignar?". Son preguntas sobre decisiones, no sobre datos.

Señales de que algo NO es variable: Si el texto te da un número sin preguntarte nada — un precio fijo, una capacidad máxima, un requerimiento nutricional — ese número es un *parámetro*. Los parámetros son datos del problema; las variables son las incógnitas que el modelo debe determinar.

Nombra cada variable con un símbolo (x_1, x_2, ...) y escribe en una oración exacta qué representa, incluyendo sus unidades.

2.2. Pregunta 2: ¿Qué quieres optimizar?

Identifica la **función objetivo**: la cantidad que mide el éxito de tu decisión, expresada en función de las variables.

Señales de maximización: "maximizar", "la mayor ganancia", "el mayor rendimiento", "lo más posible".

Señales de minimización: "minimizar", "el menor costo", "lo más barato", "el menor tiempo", "reducir al mínimo".

La función objetivo tiene siempre la forma:

$$z = c_1x_1 + c_2x_2 + \cdots$$

donde c_i es la contribución *por unidad* de la variable x_i.

Truco para encontrar c_i. Pregúntate: si produzco *una* unidad más de x_1 y todo lo demás queda igual, ¿cuánto cambia el objetivo? Esa cantidad es c_1.

2.3. Pregunta 3: ¿Qué te limita?

Identifica las **restricciones**: condiciones que el modelo debe respetar. Cada restricción corresponde a un recurso limitado o a una regla del negocio.

Señales de límite superior ($\leq$): "no puede exceder", "disponible", "capacidad máxima", "no más de", "como máximo".

Señales de mínimo obligatorio ($\geq$): "al menos", "mínimo requerido", "no puede bajar de", "debe garantizar", "tiene que ser al menos".

Señales de igualdad exacta ($=$): "exactamente", "debe ser justo", "no puede sobrar ni faltar". Son las menos comunes.

Para cada restricción, construye una tabla antes de escribir la inecuación. Cada fila de la tabla es una restricción:

Recurso	x_1	x_2	**Límite**
Recurso A	a_{11}	a_{12}	b_1
Recurso B	a_{21}	a_{22}	b_2

La restricción de la fila i es: $a_{i1}x_1 + a_{i2}x_2 \;\square\; b_i$, con el signo $\square$ que indica el texto.

2.4. Pregunta 4: ¿Pueden ser negativos?

Casi nunca. No tiene sentido producir -5 sarapes. Siempre añade:

$$x_1 \geq 0, \quad x_2 \geq 0.$$

Solo omite esta condición si el problema lo indica explícitamente (ej. una variable que representa un cambio

que puede ser positivo o negativo).

3. Ejemplo 1 — Una Panadería (Problema Claro)

Leamos un problema paso a paso, pensando en voz alta.

Texto: *"Una panadería produce pan dulce y tamales. Cada pieza de pan necesita 3 kg de harina y 1 hora de trabajo; cada tamal, 2 kg de harina y 2 horas. La semana tienen 120 kg de harina y 60 horas disponibles. La ganancia es \$5 por pan y \$8 por tamal. ¿Cuánto producir para maximizar la ganancia?"*

Pregunta 1 — ¿Qué decido? La frase "¿cuánto producir?" señala dos cantidades a decidir:

- y_1: piezas de pan dulce por semana
- y_2: tamales por semana

Pregunta 2 — ¿Qué optimizo? "Maximizar la ganancia." Cada pan aporta \$5, cada tamal \$8:

$$\max\ z = 5y_1 + 8y_2$$

Pregunta 3 — ¿Qué me limita? Hay dos recursos. La palabra "disponibles" indica $\leq$. Construyo la tabla:

Recurso	y_1	y_2	Disp.
Harina (kg)	3	2	120
Trabajo (h)	1	2	60

$$3y_1 + 2y_2 \leq 120 \qquad y_1 + 2y_2 \leq 60$$

Pregunta 4: $y_1, y_2 \geq 0$.

Resolviendo, los vértices son $(0,0)$, $(40,0)$, $(30,15)$, $(0,30)$. Evaluando z: $z(30,15) = 5(30) + 8(15) = 270$. Ese es el óptimo: 30 panes y 15 tamales por semana.

4. Errores Comunes al Formular

Estos errores los comete casi todo el mundo la primera vez. Reconócelos antes de caer en ellos.

Error 1: Confundir parámetro con variable.
"La harina cuesta \$50/kg." Ese precio es un dato del problema, no una decisión tuya. El error es tratarlo como si fuera una variable. *Señal:* si el texto te da un número sin preguntarte nada, es un parámetro.

Error 2: Equivocarse con el signo de la inecuación.
"Dispone de 120 kg" $\rightarrow \leq 120$ (no puedes usar más de lo disponible). "Pedido mínimo de 10 piezas" $\rightarrow \geq 10$ (no puedes producir menos que el pedido). Confundir estos dos casos convierte el modelo en algo completamente diferente.

Error 3: Mezclar unidades en una restricción.
Si y_1 está en piezas y y_2 en kilogramos, no puedes escribir $y_1 + y_2 \leq 100$ directamente. Los coeficientes de las variables hacen la conversión de unidades: "cada pieza de pan usa 3 kg de harina" $\rightarrow$ el coeficiente 3 convierte de piezas a kilogramos. Verifica siempre que todos los términos de una restricción tengan las mismas unidades.

Error 4: Olvidar restricciones implícitas.
El texto no siempre lista todas las restricciones; algunas se dan por entendidas en el contexto. Si el mercado solo absorbe como máximo 50 sarapes por semana, eso es una restricción aunque el texto lo mencione de pasada. Lee el problema completo buscando cualquier limitación, no solo las obvias.

Lista de verificación. Antes de escribir la primera ecuación pregúntate: (1) ¿Listé todo lo que decido? (2) ¿Hay recursos que se agotan ($\leq$)? (3) ¿Hay mínimos que cumplir ($\geq$)? (4) ¿El signo de cada restricción es el correcto? (5) ¿Las unidades son consistentes en cada término?

5. Ejemplo 2 — Un Rancho Ganadero (Minimización y $\geq$)

Este ejemplo es diferente al primero en dos aspectos clave: es *minimización* y sus restricciones son $\geq$. Así ves que las cuatro preguntas funcionan para cualquier tipo de PL.

Texto: *"Don Aurelio mezcla cada día alfalfa y maíz molido para alimentar a su ganado. Los nutricionistas le indican que cada kilogramo de ración debe contener* ***al menos 15 g de proteína*** *y* ***al menos 20 unidades de energía****. La alfalfa aporta 20 g de proteína y 15 unidades de energía por kilo; el maíz aporta 8 g de proteína y 30 unidades de energía por kilo. La alfalfa cuesta \$3/kg y el maíz \$2/kg. ¿Qué mezcla minimiza el costo diario?"*

Pregunta 1. "¿Qué mezcla?" $\rightarrow$ cuánto de cada forraje:

- a: kg de alfalfa por día
- m: kg de maíz por día

Pregunta 2. "Minimizar el costo diario." La alfalfa cuesta \$3/kg y el maíz \$2/kg:

$$\min\ c = 3a + 2m$$

Pregunta 3. "Al menos" señala $\geq$. Tabla de nutrientes:

Nutriente	a	m	Mín.
Proteína (g/kg)	20	8	15
Energía (u/kg)	15	30	20

$$20a + 8m \geq 15 \qquad 15a + 30m \geq 20$$

Pregunta 4: $a, m \geq 0$.

Observa las diferencias respecto al Ejemplo 1. (a) El objetivo es min en vez de max. (b) Las restricciones usan $\geq$ porque son mínimos obligatorios. (c) La lógica es *idéntica*: cuatro preguntas, misma tabla. En Python, solo cambia `LpMaximize` por `LpMinimize` y $\leq$ por $\geq$. La estructura del código no cambia.

6. Python vs. Excel Solver: ¿Por Qué Python?

Ambas herramientas resuelven correctamente cualquier problema de PL. La diferencia es práctica.

Excel Solver requiere capturar cada coeficiente en una celda, configurar la celda objetivo y agregar cada restricción manualmente en un cuadro de diálogo. Para 2 variables y 2 restricciones eso tarda unos 15 minutos. Para un problema con 6 variables, 8 restricciones y coeficientes que varían por alumno, puede tomar toda la hora de examen solo en captura — sin haber resuelto nada.

Python + PuLP se escribe una vez. Después, cambiar cualquier parámetro (por ejemplo, tu valor de d) significa modificar *una sola línea*. El mismo código resuelve un problema de 2 variables o de 200 variables sin cambiar la estructura.

En el examen de esta unidad, los coeficientes de la función objetivo dependen de tu d personal. En Solver tendrías que recalcular y recapturar todo desde cero. En Python:

```
d = 17  # <- cambia este numero
```

El código ajusta todo automáticamente. Esa es la ventaja real.

7. La Región Factible y sus Vértices

La figura 1 muestra la región factible del problema del examen. Las restricciones son:

$$2x_1 + x_2 \leq 100 \quad \text{(tejido)}$$

$$x_1 + 3x_2 \leq 90 \quad \text{(bordado)}$$

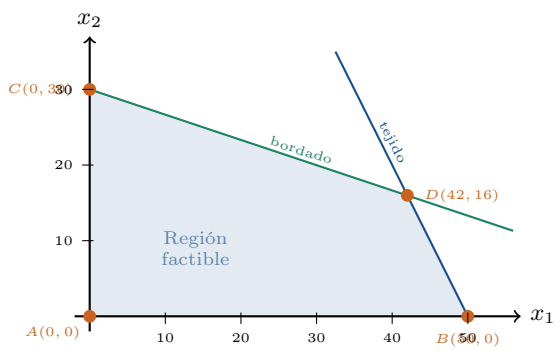

Figure 1: Región factible del examen. Los cuatro vértices son los candidatos al óptimo. Cuál gana depende de la pendiente de la función objetivo — es decir, de tu d.

Los cuatro vértices son:

Vértice	Coord.	Rest. activas
A	$(0, 0)$	ambos ejes
B	$(50, 0)$	tejido, $x_2 = 0$
C	$(0, 30)$	bordado, $x_1 = 0$
D	$(42, 16)$	tejido y bordado

Cálculo del vértice D: resuelve el sistema $2x_1 + x_2 = 100$ y $x_1 + 3x_2 = 90$ simultáneamente. Multiplica la segunda por 2 y réstala a la primera: $-5x_2 = -80 \rightarrow x_2 = 16$, luego $x_1 = 42$.

8. Curvas de Nivel y la Pendiente que Cambia

La función objetivo del examen es $z = (d+1)x_1 + (60-d)x_2$. Sus curvas de nivel son rectas con pendiente:

$$\text{pendiente} = -\frac{d+1}{60-d}$$

d pequeño (ej. $d = 5$): pendiente suave (≈ -0.11), las curvas son casi horizontales y el óptimo tiende hacia $C = (0, 30)$.

d grande (ej. $d = 25$): pendiente empinada (≈ -0.74), el óptimo tiende hacia $D = (42, 16)$ o $B = (50, 0)$.

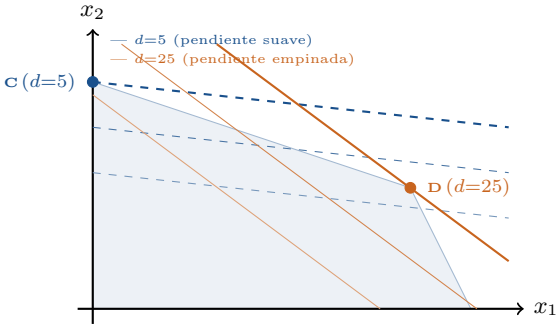

Figure 2: La misma región, dos familias de curvas de nivel. Tu valor de d determina la pendiente y, con ella, cuál vértice gana.

Idea clave. Los vértices no se mueven: siempre son A, B, C, D. Lo que cambia con d es la pendiente de las curvas de nivel. Esa pendiente determina cuál vértice queda "más alto" que todos los demás.

9. De la Fórmula al Código: Python con PuLP

PuLP es una librería de Python para PL. Su estructura replica exactamente la formulación matemática: primero las variables, luego el objetivo, luego las restricciones.

9.1. Instalación (una sola vez)

```
pip install pulp
```

9.2. La plantilla completa, línea por línea

Listing 1: Plantilla PuLP para el examen

```
from pulp import *
# Importa LpProblem, LpVariable,
# LpMaximize, LpStatus, value, etc.

# 1. TU NUMERO PERSONAL (solo cambia esto)
d = 17  # dia de tu cumpleanios

# 2. Crear el problema
# LpMaximize = maximizacion
# LpMinimize = minimizacion (Ejemplo 2)
prob = LpProblem("Manos_Durangenias",
                 LpMaximize)

# 3. Variables de decision
# lowBound=0 impone x1, x2 >= 0
x1 = LpVariable("sarapes", lowBound=0)
x2 = LpVariable("bolsas",  lowBound=0)

# 4. Funcion objetivo
# prob += [expresion]  =>  PuLP sabe que
```

```python
# debe optimizarla segun LpMaximize
prob += (d + 1)*x1 + (60 - d)*x2

# 5. Restricciones
# prob += [inecuacion], "nombre"
# El nombre aparece en la salida
prob += 2*x1 +   x2 <= 100, "tejido"
prob +=   x1 + 3*x2 <=  90, "bordado"

# 6. Resolver con el solver incluido
prob.solve(PULP_CBC_CMD(msg=0))

# 7. Imprimir resultados
print("Estado:", LpStatus[prob.status])
print(f"x1 = {x1.varValue:.1f} sarapes")
print(f"x2 = {x2.varValue:.1f} bolsas")
print(f"z  = {value(prob.objective):.2f}")

# 8. Estado de cada restriccion
for nombre, c in prob.constraints.items():
    s = c.slack
    if abs(s) < 1e-6:
        estado = "ACTIVA (recurso agotado)"
    else:
        estado = f"holgura = {s:.1f} h"
    print(f"  {nombre}: {estado}")
```

9.3. Cómo leer la salida

Una corrida típica (con $d = 17$) muestra:

```
Estado:  Optimal
x1 = 42.0 sarapes
x2 = 16.0 bolsas
z  = 1652.00
tejido:  ACTIVA (recurso agotado)
bordado:  ACTIVA (recurso agotado)
```

El estado `Optimal` confirma que se encontró la solución óptima. Los valores $x_1 = 42$, $x_2 = 16$ son el plan de producción. Ambas restricciones activas significan que se usan exactamente las 100 horas de tejido y las 90 de bordado — sin desperdiciar ninguna.

10. Restricciones Activas y Holguras

Definición : Holgura de una restricción

Si la restricción es $ax_1 + bx_2 \leq c$, la **holgura** en el óptimo (x_1^*, x_2^*) es:

$$s = c - (ax_1^* + bx_2^*) \geq 0.$$

$s = 0$: restricción **activa** (recurso totalmente agotado).
$s > 0$: restricción **inactiva** (sobran s unidades del recurso).

¿Por qué importa esto en la práctica?

Una restricción activa es un **cuello de botella**: el factor que impide producir más. Si la cooperativa quisiera aumentar su ganancia, tendría que relajar alguna restricción activa — contratar más bordadoras, conseguir más horas de tejido. Ampliar un recurso que tiene holgura no cambia nada: el límite ya no está ahí.

Regla práctica. Si quieres aumentar la ganancia, solo vale la pena invertir en ampliar un recurso que esté *activo*. Ampliar un recurso con holgura no mueve el óptimo.

10.1. Agregar una restricción nueva al código

Si en la Parte IV aparece una restricción adicional (ej. "pedido mínimo de 12 sarapes"), añades *una línea* antes de `prob.solve()`:

```python
# Restriccion adicional (Parte IV)
prob += x1 >= 12, "pedido_minimo"
```

Re-ejecutas y PuLP recalcula el óptimo automáticamente.

11. Tabla de Traducción Rápida

En el problema	En Python
"maximizar"	`LpMaximize`
"minimizar"	`LpMinimize`
Variable x_1	`LpVariable(...)`
No negatividad	`lowBound=0`
$c_1x_1 + c_2x_2$	`prob += c1*x1 + c2*x2`
$ax_1 + bx_2 \leq k$	`prob += a*x1+b*x2 <= k`
$ax_1 + bx_2 \geq k$	`prob += a*x1+b*x2 >= k`
$x_1 = k$ (exacto)	`prob += x1 == k`
Resolver	`prob.solve(...)`
Valor óptimo x_1^*	`x1.varValue`
Valor de z^*	`value(prob.objective)`
Holgura	`c.slack`

Síntesis final. Formular un problema de PL es un ejercicio de traducción: de palabras a variables, de recursos a restricciones, de objetivos a funciones. Las señales en el texto — "maximizar", "al menos", "disponible", "no puede exceder" — te dicen exactamente qué elemento tienes frente a ti. Una vez construida la formulación, Python la resuelve en milisegundos. La tabla de recursos es tu herramienta central: si la llenas correctamente, las ecuaciones se escriben solas. El código no es magia: es la formulación matemática escrita en un idioma que la computadora entiende. Si entiendes la fórmula, entiendes el código.